Manuel d'utilisation de deux scripts d'automatisation pour MadDM

`scan_relic_paral_py3.py` et `scan4_heatmap.py`

Objet. Ce document décrit l'arborescence des commandes, la logique de fonctionnement et les sorties produites par deux scripts Python destinés à automatiser des études de matière noire avec MadDM. Le premier pilote les scans de densité relique et l'extraction d'observables de détection directe et indirecte. Le second post-traite les fichiers CSV issus du scan 4 afin de produire des cartes, des re-scalings en couplage et des courbes iso-Ωh^2.

Ruben CLAIE ruben.claie-ferrachat@etu.sorbonne-universite.fr

Lucie DESPEYROUX lucie.despeyroux@etu.sorbonne-universite.fr

Attention. Ne vaut actuellement que pour l'étude d'un modèle en t-channel avec un seul couplage au quark droit bottom. Si vous étudiez un autre couplage, changez les mentions de bbx et les numéros correspondant à votre modèle (dans les argparse).

Seulement testé avec le modèle suivant : DMSimpt_v2_0_LOmassive (BFuks) avec une *restrict card* appropriée. Fait pour la version MG5_aMC_v2_9_27 et la version dev de MadDM.

Mars 2026

Résumé

MadDM est un outil numérique automatisé construit dans l'écosystème MG5_aMC pour l'étude phénoménologique de la matière noire. Il permet notamment de calculer la densité relique thermique, des observables de détection directe et des observables de détection indirecte dans des modèles génériques importés sous forme UFO. Les publications *MadDM v2.0* et *MadDM v3.2* montrent l'extension progressive du code vers la détection directe, puis vers des observables de détection indirecte plus avancées, y compris certains processus induits à une boucle. Le présent document ne décrit pas l'ensemble de MadDM, mais deux scripts d'automatisation qui l'encapsulent pour réaliser des scans systématiques et en visualiser les résultats.

1 Portée du document

Les deux scripts documentés remplissent des rôles complémentaires :

— `scan_relic_paral_py3.py` : lancement de points MadDM, organisation des scans, création automatique des projets MadDM, extraction de la densité relique et des limites de détection, écriture des CSV, production de figures et ajustements en loi de puissance pour les scans 1 à 3.

— `scan4_heatmap.py` : lecture des CSV du scan 4, interpolation bidimensionnelle, visualisation des régions favorisées ou exclues et re-scaling de Ωh^2 lorsqu'on change le couplage λ.

Le flux de travail nominal est donc :

```
scan_relic_paral_py3.py  --->  CSV scan4_omega_map_lambda_*.csv
                                  |
                                  v
                            scan4_heatmap.py
                                  |
                                  v
                            cartes / courbes PNG
```

2 Rappel minimal sur MadDM

Au niveau conceptuel, MadDM prend un modèle particulaire, identifie le ou les candidats de matière noire, génère les processus utiles, puis calcule des observables cosmologiques et astrophysiques. Dans le cadre des deux scripts ici étudiés, trois blocs physiques sont exploités :

1. **Densité relique** : extraction de Ωh^2 via la sortie `Omegah2`.
2. **Détection directe** : extraction de sections efficaces nucléon–matière noire, séparées en contributions SI/SD et proton/neutron via les clés `SigmaN_SI_p`, `SigmaN_SI_n`, `SigmaN_SD_p` et `SigmaN_SD_n`.
3. **Détection indirecte** : extraction de prédictions et de limites pour `xsxs_bbx` et `Total_xsec`.

Dans ces scripts, la logique de décision est très simple : un point est considéré comme *exclu* pour un observable donné quand une limite supérieure numérique existe ($\texttt{UL} \neq -1$) et que la prédiction dépasse cette limite.

3 Script 1 : `scan_relic_paral_py3.py`

3.1 Rôle général

Ce script pilote toute la campagne de calcul. Il :

1. vérifie la présence de `bin/maddm` dans un répertoire MG5_aMC ;
2. crée si besoin un projet MadDM ;
3. génère des scripts d'entrée MadDM temporaires ;
4. exécute MadDM point par point ;
5. extrait les observables depuis la sortie standard ou, en secours, depuis les fichiers du projet ;

6. lance jusqu'à quatre scans ;
7. produit des CSV et des figures PNG ;
8. supprime à la fin les projets MadDM qu'il a créés lui-même pendant l'exécution.

3.2 Arborescence des commandes

```
python scan_relic_paral_py3.py --mg5 <chemin_MG5> [options globales]

|-- Configuration du modele
|   |-- --model <nom_UFO>
|   |-- --project <nom_projet_MadDM>
|   `-- --dmname <nom_particule_DM>
|
|-- Parametres de carte
|   |-- --dm-pdg <entier>
|   |-- --med-pdg <entier>
|   |-- --coup-block <bloc_SLHA>
|   |-- --coup-i <indice_i>
|   `-- --coup-j <indice_j>
|
|-- Gestion des sorties
|   `-- --outdir <repertoire_sortie>
|
`-- Selection des scans
    `-- --scans {all | 1,2,3,4}
        |-- Scan 1 : Omega h2 = f(My)
        |   |-- --scan1-mx <Mx_fixe>
        |   |-- --scan1-my <a_to_b>
        |   |-- --scan1-step <pas_GeV>
        |   `-- --scan1-lambda <lambda_fixe>
        |
        |-- Scan 2 : Omega h2 = f(Mx)
        |   |-- --scan2-my <My_fixe>
        |   |-- --scan2-mx <a_to_b>
        |   |-- --scan2-step <pas_GeV>
        |   `-- --scan2-lambda <lambda_fixe>
        |
        |-- Scan 3 : Omega h2 = f(lambda)
        |   |-- --scan3-mx <Mx_fixe>
        |   |-- --scan3-my <My_fixe>
        |   |-- --scan3-lambda <a_to_b>
        |   `-- --scan3-n <nombre_de_points>
        |
        `-- Scan 4 : Omega h2(Mx,My) + DD + ID
            |-- --scan4-step <pas_GeV>
            |-- --scan4-lambda <lambda_fixe>
            |-- --scan4-mx <a_to_b>
            `-- --scan4-my <a_to_b>
```

3.3 Commandes MadDM générées automatiquement

Le script encapsule MadDM, mais il faut savoir quelles commandes internes il lui envoie.

Initialisation d'un projet. Quand le projet n'existe pas, la fonction `ensure_project()` envoie la séquence suivante :

```
import model <model>
```

```
define darkmatter <dmname>
generate relic_density
"add direct_detection" # Comment if you don't use scan 4 to save time
"add indirect_detection" # Comment if you don't use scan 4 to save time
output <project>
quit
```

Cette séquence :

— importe le modèle UFO ;

— déclare la particule de matière noire ;

— génère le calcul de densité relique ;

— génère les calculs liés au détections DD/ID (très longs !) ;

— crée un projet MadDM autonome dans le répertoire MG5_aMC.

Évaluation d'un point. Pour un point de scan, la fonction `maddm_point_script()` fabrique :

```
launch <project>
set param_card mass <dm_pdg> <m_dm>
set param_card mass <med_pdg> <m_med>
set param_card <coup_block> <i> <j> <lambda>
4 # Menu interactif (ID): passage sigmav -> flux_source
4 # Menu interactif (ID): passage flux_source -> flux_earth
quit
```

Les trois commandes `set param_card` modifient explicitement la masse de la matière noire, la masse du médiateur et un couplage dans la carte de paramètres. **Dans l'état actuel du script, commenter les ajouts des détections est vivement recommandée si le scan 4 n'est pas utilisé pour éviter les calculs longs !**. Par défaut, le flux est réglé sur `flux_earth` par l'envoi de deux 4.

3.4 Description physique des quatre scans

Scan 1 : Ωh^2 en fonction de M_y

Paramètres fixés par l'utilisateur : `-scan1-mx` pour M_x et `-scan1-lambda` pour λ.
Balayage : `-scan1-my` pour l'intervalle en M_y et `-scan1-step` pour le pas.
Par défaut, le script utilise $M_x = 1000$ GeV, $\lambda = 1$, $M_y \in [1100, 4100]$ GeV et $\Delta M_y = 250$ GeV.
Sorties :

— `scan1_omega_vs_mMed.csv`

— `scan1_omega_vs_mMed.png`

— `scan1_omega_vs_mMed_loglog.png`

Le script tente aussi un ajustement en loi de puissance avec `scipy.optimize.curve_fit`.

Scan 2 : Ωh^2 en fonction de M_x

Paramètres fixés par l'utilisateur : `-scan2-my` pour M_y et `-scan2-lambda` pour λ.
Balayage : `-scan2-mx` pour l'intervalle en M_x et `-scan2-step` pour le pas.
Par défaut, le script utilise $M_y = 2000$ GeV, $\lambda = 1$, $M_x \in [100, 1900]$ GeV et $\Delta M_x = 100$ GeV.
Sorties :

— `scan2_omega_vs_mDM.csv`

— `scan2_omega_vs_mDM.png`

— `scan2_omega_vs_mDM_loglog.png`

Comme pour le scan 1, l'ajustement est maintenant réalisé avec `scipy.optimize.curve_fit`.

Scan 3 : Ωh^2 en fonction du couplage λ

Paramètres fixés par l'utilisateur : `-scan3-mx` pour M_x et `-scan3-my` pour M_y.
Balayage log-uniforme : `-scan3-lambda`, avec `-scan3-n` points.
Par défaut, le script utilise $M_x = 1000$ GeV, $M_y = 2000$ GeV et $\lambda \in [10^{-3}, 10]$.
Sorties :

— `scan3_omega_vs_couplage.csv`

— `scan3_omega_vs_couplage_loglog.png`

Le fit en loi de puissance est lui aussi réalisé avec `scipy.optimize.curve_fit`.

Scan 4 : carte de Ωh^2 dans le plan (M_x, M_y) avec DD/ID

Paramètre fixé : `-scan4-lambda` pour λ. Balayage sur une grille (M_x, M_y) avec condition cinématique imposée dans le script :

$$M_y > M_x.$$

Ce scan est parallélisé sur `cpu_count()` processus. Le script crée un projet MadDM distinct par processus, ce qui évite les collisions d'écriture. La sortie principale est un CSV contenant :

— M_x, M_y, Ωh^2 ;

— les prédictions de DD (`SigmaN_*`) et leurs limites supérieures ;

— les prédictions de ID (`xsxs_bbx`, `Total_xsec`) et leurs limites supérieures.

3.5 Table détaillée des options

Option	Défaut	Fonction
`-mg5`	obligatoire	Chemin vers le répertoire MG5_aMC contenant l'exécutable `bin/maddm`. Sans cette option, le script ne peut rien lancer.
`-model`	`DMSimpt_v2_0_LOmassive-F3S_br`	Nom du modèle UFO à importer dans MadDM. Il est utilisé lors de la création initiale du projet.
`-project`	`scan_bottom_RD`	Nom de base du projet MadDM. Pour le scan 4 parallèle, le script dérive automatiquement `<project>_1`, `<project>_2`, etc.
`-dmname`	`xs`	Nom de la particule de matière noire utilisé dans `define darkmatter`.
`-outdir`	`scan_outputs`	Répertoire dans lequel sont déposés les CSV et les figures PNG.
`-scans`	`all`	Sélection des scans à exécuter. Les valeurs acceptées sont `all` ou une liste telle que `1,3,4`.
`-dm-pdg`	`51`	Code PDG de la matière noire dans la carte de masses. Sert à modifier `mass <dm-pdg> <m_dm>`.
`-med-pdg`	`5920005`	Code PDG du médiateur dans la carte de masses.
`-coup-block`	`dmf3d`	Nom du bloc SLHA dans lequel est lu le couplage à modifier.
`-coup-i`	`3`	Premier indice de la matrice de couplage dans le bloc SLHA.
`-coup-j`	`3`	Second indice de la matrice de couplage dans le bloc SLHA.
`-scan1-mx`	`1000.0`	Masse fixe M_x en GeV pour le scan 1.
`-scan1-my`	`1100to4100`	Intervalle balayé en M_y pour le scan 1. Le script accepte les syntaxes `a to b`, `a:b`, `a,b` ou `a..b`.
`-scan1-step`	`250.0`	Pas en GeV utilisé pour construire le balayage en M_y du scan 1.
`-scan1-lambda`	`1.0`	Valeur fixe du couplage λ pour le scan 1.
`-scan2-my`	`2000.0`	Masse fixe M_y en GeV pour le scan 2.

Option	Défaut	Fonction
`-scan2-mx`	`100to1900`	Intervalle balayé en M_x pour le scan 2, avec la même syntaxe souple que pour `-scan1-my`.
`-scan2-step`	`100.0`	Pas en GeV utilisé pour construire le balayage en M_x du scan 2.
`-scan2-lambda`	`1.0`	Valeur fixe du couplage λ pour le scan 2.
`-scan3-mx`	`1000.0`	Masse fixe M_x en GeV pour le scan 3.
`-scan3-my`	`2000.0`	Masse fixe M_y en GeV pour le scan 3.
`-scan3-lambda`	`1e-3to10`	Intervalle de balayage du couplage pour le scan 3. Cet intervalle est échantillonné en log-espace.
`-scan3-n`	`30`	Nombre de points du scan log-uniforme en couplage pour le scan 3.
`-scan4-step`	`50.0`	Pas de la grille du scan 4 en GeV. Si `-scan4-mx` ou `-scan4-my` ne sont pas fournis, ce pas est appliqué à la grille par défaut.
`-scan4-lambda`	`1.0`	Valeur fixe du couplage pour le scan 4. Cette valeur est aussi encodée dans le nom du CSV de sortie.
`-scan4-mx`	`None`	Intervalle de balayage de M_x pour le scan 4. Les syntaxes `5to4000`, `5:4000`, `5,4000` et `5 to 4000` sont acceptées.
`-scan4-my`	`None`	Intervalle de balayage de M_y pour le scan 4 avec la même logique de syntaxe que `-scan4-mx`.

3.6 Règles d'extraction et de robustesse

Le script ne se contente pas de lire la sortie standard. Il applique deux stratégies :

1. **Extraction directe depuis stdout**. Des expressions régulières recherchent `Omegah2 = ...`, les lignes `SigmaN_* = [prediction, UL]` et les lignes `xsxs_bbx = [prediction, UL]` ou `Total_xsec = [prediction, UL]`.
2. **Plan B par lecture du projet**. Si la sortie standard ne contient pas l'information attendue, le script parcourt les fichiers du projet MadDM et récupère la valeur la plus récente trouvée.

Deux détails importants pour l'interprétation :

— Dans les scans 1 et 2 seulement, une densité relique non finie ou non positive est remplacée par 1.0 via `omega_default_if_bad`. Ce remplacement n'est *pas* appliqué dans les scans 3 et 4.

— Une limite supérieure égale à -1 signifie *pas de limite disponible*. Dans ce cas, le point n'est ni marqué comme autorisé ni comme exclu pour l'observable considéré.

3.7 Exemples d'utilisation

Exécuter tous les scans.

```
python scan_relic_paral_py3.py \
  --mg5 /chemin/vers/MG5_aMC \
  --outdir scan_outputs
```

Personnaliser le scan 1.

```
python scan_relic_paral_py3.py \
  --mg5 /chemin/vers/MG5_aMC \
  --scans 1 \
  --scan1-mx 750 \
  --scan1-my 900to3000 \
  --scan1-step 150 \
  --scan1-lambda 0.8
```

Personnaliser le scan 3.

```
python scan_relic_paral_py3.py \
  --mg5 /chemin/vers/MG5_aMC \
  --scans 3 \
  --scan3-mx 800 \
  --scan3-my 1600 \
  --scan3-lambda 1e-4to3 \
  --scan3-n 40
```

Lancer uniquement le scan 4 sur une fenêtre restreinte.

```
python scan_relic_paral_py3.py \
  --mg5 /chemin/vers/MG5_aMC \
  --scans 4 \
  --scan4-lambda 1 \
  --scan4-step 100 \
  --scan4-mx 100to2000 \
  --scan4-my 200to4000
```

4 Script 2 : `scan4_heatmap.py`

4.1 Rôle général

Ce script lit un CSV du scan 4 et produit des visualisations. Il peut fonctionner en trois modes logiques :

1. **mode carte** : représentation de Ωh^2 avec ou sans hachures de contraintes DD/ID ;
2. **mode re-scaling** : recalcul de Ωh^2 pour d'autres couplages via la loi $\Omega \propto \lambda^{-4}$;
3. **mode iso-courbes** : tracé des lignes de niveau $\Omega h^2 =$ constante.

4.2 Arborescence des commandes

```
python scan4_heatmap.py <csvfile> [options]

|-- Modes de carte
|   |-- --all
|   `-- --all2
|
|-- Reglages visuels
|   |-- --graph2
|   |-- --nointerpolation
|   `-- --interp-n <entier>
|   |-- --zoom-mx <a_to_b>
|   `-- --zoom-my <a_to_b>
|
`-- Modes analytiques
    |-- --lambdas <liste>
    `-- --iso_omega <liste>
```

En pratique, le script vise les usages suivants :

— `-all` ou `-all2` pour une carte de Ωh^2 ;

— `-lambdas` pour re-scalé Ωh^2 à partir d'un CSV de référence ;

— `-iso_omega` pour des courbes iso-Ωh^2, avec éventuellement `-lambdas` en complément.

4.3 Formats de données attendus

Le CSV d'entrée doit contenir au minimum :

```
mDM_GeV, mMed_GeV, Omega_h2
```

et, pour activer les hachures DD/ID :

```
SigmaN_SI_p, UL_SigmaN_SI_p
SigmaN_SI_n, UL_SigmaN_SI_n
SigmaN_SD_p, UL_SigmaN_SD_p
SigmaN_SD_n, UL_SigmaN_SD_n
xsxs_bbx,   UL_xsxs_bbx
Total_xsec, UL_Total_xsec
```

4.4 Logique physique et graphique

Interpolation. Par défaut, la carte de fond est interpolée en deux dimensions sur le plan (M_y, M_x) à l'aide d'une triangulation linéaire. Les points sont restreints à la région $M_y > M_x$. L'option `-nointerpolation` désactive cette étape et trace uniquement la grille brute issue du CSV.

Décision d'exclusion. La règle codée est :

$$\text{exclu} \iff (\text{UL} \neq -1) \text{ et } (\text{valeur} > \text{UL}).$$

Les hachures sont regroupées ainsi :

— DD SI : fusion de `SigmaN_SI_p` et `SigmaN_SI_n` ;

— DD SD : fusion de `SigmaN_SD_p` et `SigmaN_SD_n` ;

— ID `xsxs_bbx` ;

— ID `Total_xsec`.

Re-scaling en couplage. Quand on demande `-lambdas`, le script suppose :

$$\Omega(\lambda) = \Omega(\lambda_0) \left(\frac{\lambda_0}{\lambda}\right)^4.$$

La valeur λ_0 est déduite du nom du fichier CSV via le motif `lambda_<valeur>`.

4.5 Table détaillée des options

Option	Défaut	Fonction
`csvfile`	obligatoire	Fichier CSV issu du scan 4. C'est la seule entrée positionnelle du script.
`-all`	`False`	Produit une carte standard dans le plan (M_y, M_x), avec Ωh^2 en fond et, si présentes, les contraintes DD/ID sous forme de hachures.
`-all2`	`False`	Produit une carte alternative en échelles log-log avec l'axe horizontal M_x et l'axe vertical $(M_y/M_x - 1)$.
`-graph2`	`False`	Remplace l'échelle de gris centrée autour de $\Omega h^2 \approx 0.12$ par une échelle simple allant du gris clair au gris foncé entre le minimum et le maximum des données finies.

Option	Défaut	Fonction
`-nointerpolation`	`False`	Désactive l'interpolation bidimensionnelle et utilise uniquement les valeurs discrètes du CSV comme fond de carte.
`-lambdas`	`None`	Liste de nouvelles valeurs du couplage λ, séparées par des virgules, utilisée pour re-scalé Ωh^2 selon la loi en λ^{-4}.
`-iso_omega`	`None`	Liste de niveaux de Ωh^2 pour générer des courbes iso-Ωh^2 indépendamment des cartes standard.
`-interp-n`	`350`	Résolution de la grille d'interpolation ; la grille calculée est de taille $n_x = n_y =$`-interp-n`.
`-zoom-mx`	`None`	Intervalle de zoom appliqué à la grandeur portée par l'axe associé à M_x dans la figure sauvegardée. Les syntaxes `a to b`, `a:b`, `a,b` et `a..b` sont acceptées.
`-zoom-my`	`None`	Intervalle de zoom appliqué à la grandeur portée par l'axe associé à M_y dans la figure sauvegardée. Les mêmes syntaxes souples que pour `-zoom-mx` sont acceptées.

4.6 Sorties produites

Selon le mode, le script écrit typiquement :

— `scan4_omega_map_lambda_<tag>.png` ;

— `scan4_omega_map_lambda_<tag>_all2.png` ;

— `scan4_iso_omega_lambda_<tag>.png`.

Quand une interpolation est effectivement utilisée, le script imprime aussi en fin d'exécution un contrôle rapide sur trois points aléatoires du CSV, avec comparaison valeur brute vs valeur interpolée et erreur relative.

4.7 Exemples d'utilisation

Carte standard avec hachures DD/ID.

```
python scan4_heatmap.py scan4_omega_map_lambda_1.csv --all
```

Même carte, mais dans les variables M_x et $(M_y/M_x - 1)$.

```
python scan4_heatmap.py scan4_omega_map_lambda_1.csv --all2
```

Même carte sans interpolation.

```
python scan4_heatmap.py scan4_omega_map_lambda_1.csv --all --nointerpolation
```

Re-scaling de la densité relique pour plusieurs couplages.

```
python scan4_heatmap.py scan4_omega_map_lambda_1.csv --lambdas=0.5,1,2,10
```

Courbes iso-Ωh^2.

```
python scan4_heatmap.py scan4_omega_map_lambda_1.csv --iso_omega=0.08,0.12,0.20
```

5 Conseils pratiques d'utilisation

5.1 Chaîne minimale de travail

Une utilisation rationnelle des deux scripts est :

1. lancer `scan_relic_paral_py3.py` pour générer le CSV du scan 4 ;
2. vérifier que le nom du CSV encode bien la valeur de λ utilisée ;
3. lancer `scan4_heatmap.py` sur ce CSV ;
4. choisir `-all` ou `-all2` selon le plan de visualisation voulu ;
5. utiliser `-lambdas` seulement si l'hypothèse $\Omega \propto \lambda^{-4}$ est physiquement acceptable dans le régime étudié.

5.2 Points de vigilance

— Le scan 4 impose $M_y > M_x$. Aucun point ne sera calculé en dehors de cette région.

— Le script 1 nettoie automatiquement les projets MadDM qu'il a créés lui-même pendant l'exécution. Si un projet existait déjà avant le lancement, il n'est pas supprimé.

— Les $\Omega h^2 = -1.0$ ne sont pas pris en compte lors des fits pour les scans 1 à 3, mais leurs apparitions ne doit pas être ignorées.

— Les options `-all`, `-all2`, `-lambdas` et `-iso_omega` ne sont pas strictement déclarées comme mutuellement exclusives dans `argparse`. Il faut donc respecter les usages logiques présentés dans ce manuel.

6 Conclusion

Les deux scripts forment une chaîne cohérente d'automatisation autour de MadDM. Le premier encapsule la préparation d'un projet, la modification de la carte de paramètres, le balayage des paramètres et l'agrégation des observables dans des fichiers de sortie exploitables. Le second transforme les sorties du scan 4 en objets visuels directement interprétables, en ajoutant l'interpolation, les hachures d'exclusion et le re-scaling en couplage. Pour un usage fiable, il faut garder à l'esprit les conventions internes du code : condition $M_y > M_x$, et statut -1 signifiant l'absence de limite expérimentale, une erreur de calcul (non-sens physique ?).

Références

[1] M. Backovic, K. Kong, A. Martini, O. Mattelaer and G. Mohlabeng, *Direct Detection of Dark Matter with MadDM v.2.0*, Phys. Dark Univ. 9–10 (2015) 37–50.

[2] C. Arina, J. Heisig, F. Maltoni, D. Massaro and O. Mattelaer, *Indirect dark-matter detection with MadDM v3.2 – Lines and Loops*, Eur. Phys. J. C 83 (2023) 241.

[3] BFuks, *DMSimpt_v2_0_LOmassive (GitHub repository)*, https://github.com/BFuks/DMSimpt/tree/main/DMSimpt_v2_0_LOmassive.

[4] mg5amcnlo, *MG5_aMC_v2_9_27 (GitHub release)*, https://github.com/mg5amcnlo/mg5amcnlo/releases/tag/v2.9.27.

[5] mg5amcnlo, *MadDM dev branch (GitHub)*, https://github.com/mg5amcnlo/maddm/tree/dev.